Elastic SIEM Home Lab

Fleet Endpoint Deployment, Sysmon & Auditd Monitoring, Brute-Force Attack Simulation and Detection Engineering

Pavan Kumar | Author

1. Project Overview

This project involved building a three-endpoint Security Information and Event Management (SIEM) home lab using the Elastic Stack (Elasticsearch, Kibana, Fleet Server and Elastic Agent), onboarding a real Windows physical machine and a Linux virtual machine as monitored endpoints, and simulating credential brute-force activity against both. The objective was not simply to run an attack tool and capture a single alert, but to deploy and correctly configure the underlying agent fleet, diagnose and resolve the infrastructure problems that arose along the way, and validate every detection directly against live data in Kibana Discover.

The lab covers four areas: (1) SIEM infrastructure deployment and Fleet-based endpoint onboarding across Linux and Windows, (2) endpoint monitoring configuration — Sysmon on Windows and auditd on Linux, (3) attack simulation and detection validation for SSH brute-force (MITRE ATT&CK T1110) and an attempted RDP brute-force, and (4) a substantial body of infrastructure troubleshooting — certificate trust, DHCP address changes, credential rotation and output misrouting — that is documented in detail because it consumed a comparable share of effort to the detection work itself.

One planned use case, RDP brute-force detection via Sysmon, could not be completed exactly as scoped: the physical Windows machine used as the monitored endpoint runs Windows 10 Home Single Language, which does not support hosting inbound RDP sessions at the operating-system level. This is documented honestly in Section 8, together with the substitute validation method used, rather than omitted or misrepresented as a successful attack detection.

2. Lab Architecture

Component	Role
elk-server (Ubuntu Server 22.04, VirtualBox VM)	Elasticsearch + Kibana + Fleet Server. Central SIEM manager; receives and indexes all endpoint data and runs every Discover query used for detection.
linux-agent (Ubuntu 22.04, VirtualBox VM)	Monitored Linux endpoint. Runs the Elastic Agent under linux-agent-policy, with auditd watching /etc/passwd and /etc/shadow for tampering.
windows-agent “BATMAN” (Windows 10 Home Single Language, physical machine)	Monitored Windows endpoint, not a VM. Runs the Elastic Agent under hex-security-windows-policy, with the Windows integration's Sysmon Operational channel enabled.
attacker (Kali Linux, VirtualBox VM)	Attacker platform. Used to run Hydra against linux-agent (SSH) and windows-agent (RDP).

All four machines were placed on the same private subnet so that agent-to-Fleet-Server and attacker-to-endpoint traffic would behave exactly as it would in a real deployment. The subnet address range changed

twice over the course of the project — first from a mobile-hotspot DHCP lease renewal, later from a full migration to a home Wi-Fi router — and each change is discussed in Section 6, since it directly caused the majority of the infrastructure incidents documented there.

3. SIEM Deployment and Log Source Onboarding

3.1 Elastic Stack Installation

Elasticsearch, Kibana and the Elastic Agent tooling (version 9.4.3) were installed on elk-server via the official Elastic apt repository, and both core services were enabled and verified active:

```
sudo apt update && sudo apt install -y apt-transport-https gnupg2 curl
sudo systemctl enable --now elasticsearch
sudo systemctl enable --now kibana
sudo systemctl status elasticsearch kibana
```

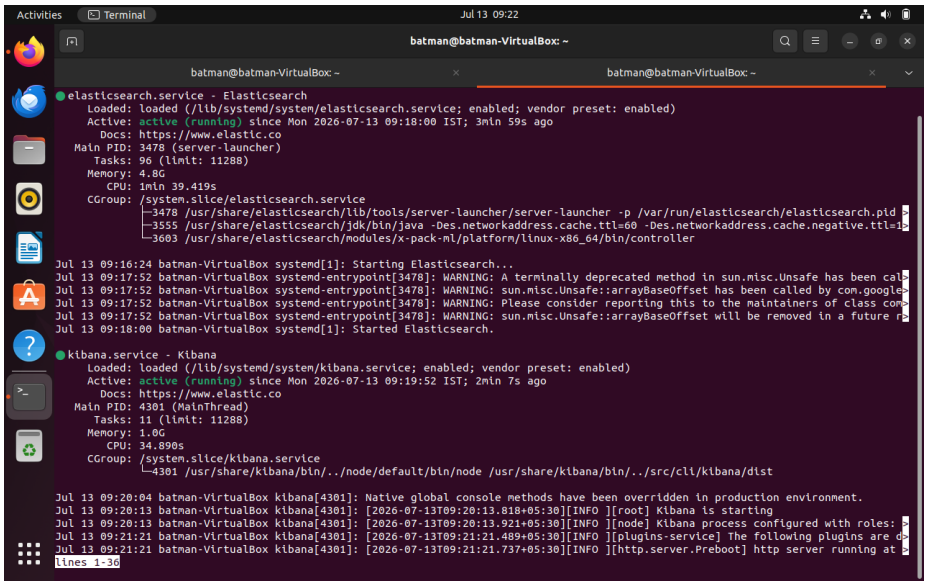

Figure 1. systemctl status confirming elasticsearch.service and kibana.service both active (running) on elk-server.

3.2 Kibana Access and Credential Setup

Kibana was reached from a browser at the elk-server address on port 5601, confirming the stack was serving its core modules:

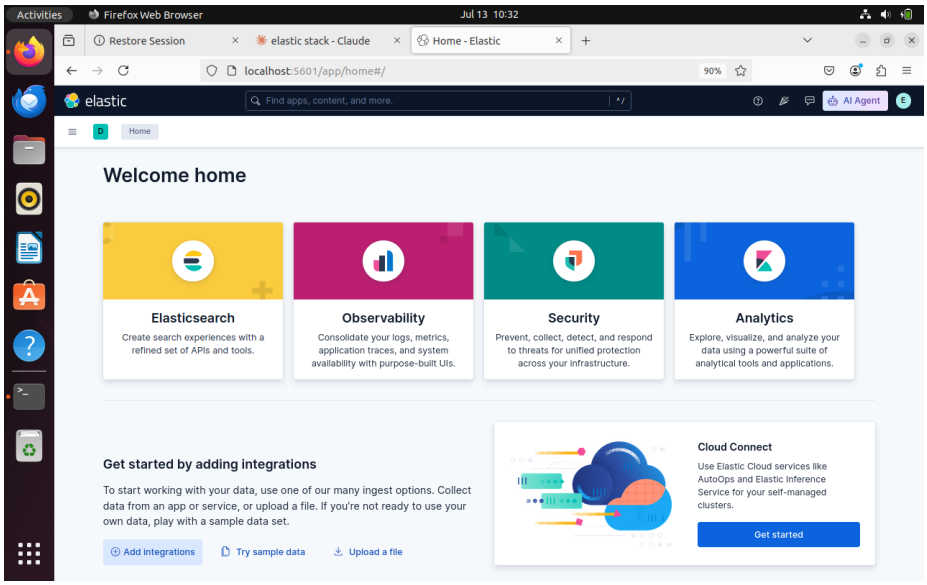

Figure 2. Kibana "Welcome home" page after first successful login, confirming Elasticsearch, Observability, Security and Analytics modules were available.

The elastic superuser and kibana_system service-account passwords were generated with Elasticsearch's built-in reset tool, and the new kibana_system value was written into Kibana's keystore so Kibana could authenticate to Elasticsearch:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u kibana_system
--url https://localhost:9200
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic --url
https://localhost:9200
cd /usr/share/kibana && sudo bin/kibana-keystore add elasticsearch.password
```

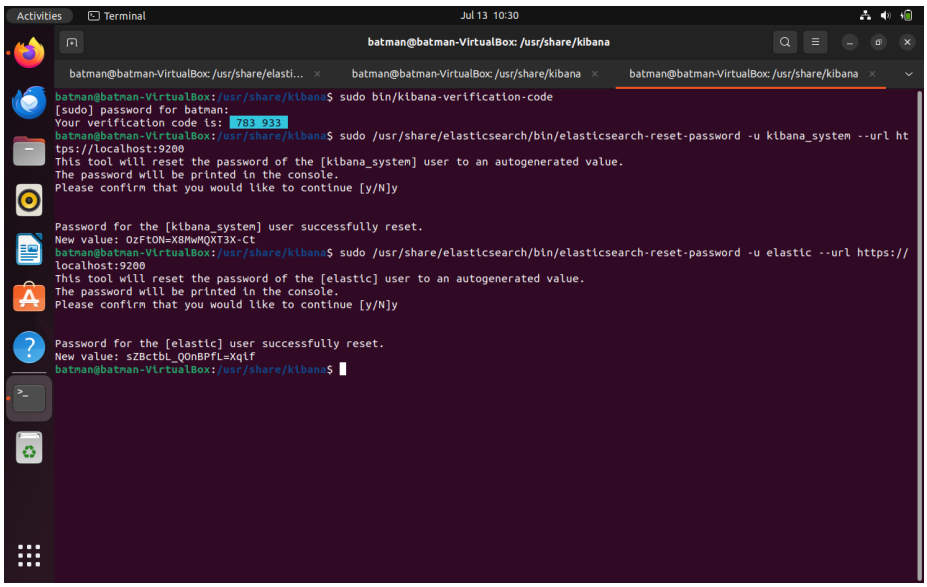

Figure 3. Terminal output resetting the kibana_system and elastic passwords and generating a Kibana verification code.

3.3 Fleet Server Deployment

Fleet Server, the control-plane component through which every endpoint agent is managed, was installed directly on elk-server via Kibana's Fleet → Settings → Add Fleet Server workflow. Once healthy, the Fleet Agents view showed exactly one agent — Fleet Server itself — prior to any endpoint enrollment:

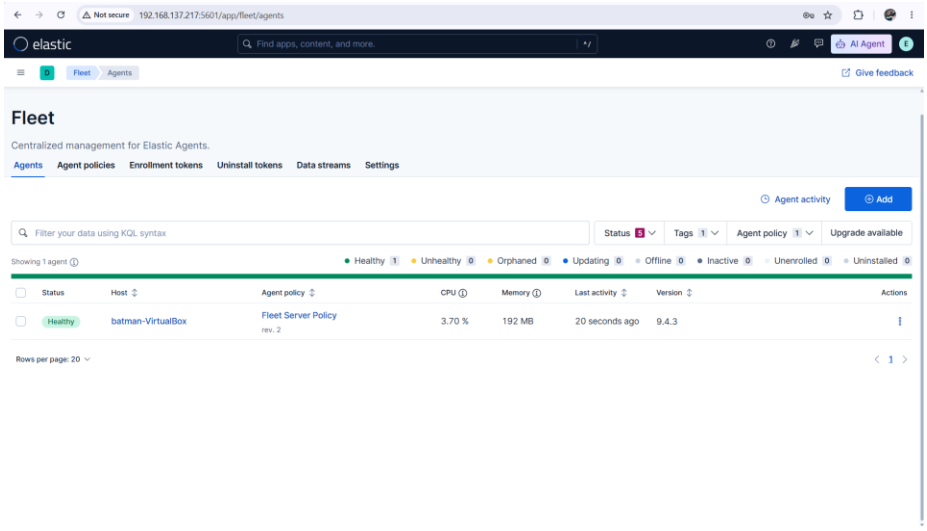

Figure 4. Fleet → Agents showing a single Healthy agent, batman-VirtualBox, under Fleet Server Policy, prior to endpoint enrollment.

4. Endpoint Agent Enrollment

4.1 Windows Agent Enrollment

A dedicated policy, hex-security-windows-policy, was created, and the Elastic Agent was installed on the Windows host using the command generated by Kibana. The first attempt failed with an x509 certificate-trust error, since Elasticsearch's auto-generated TLS certificate only trusts the address it had at install time; adding the —insecure flag — appropriate for a lab, not for production — resolved it:

```
.\elastic-agent.exe install --url=https://<elk-server-ip>:8220 --enrollment-
token=<token> --insecure
```

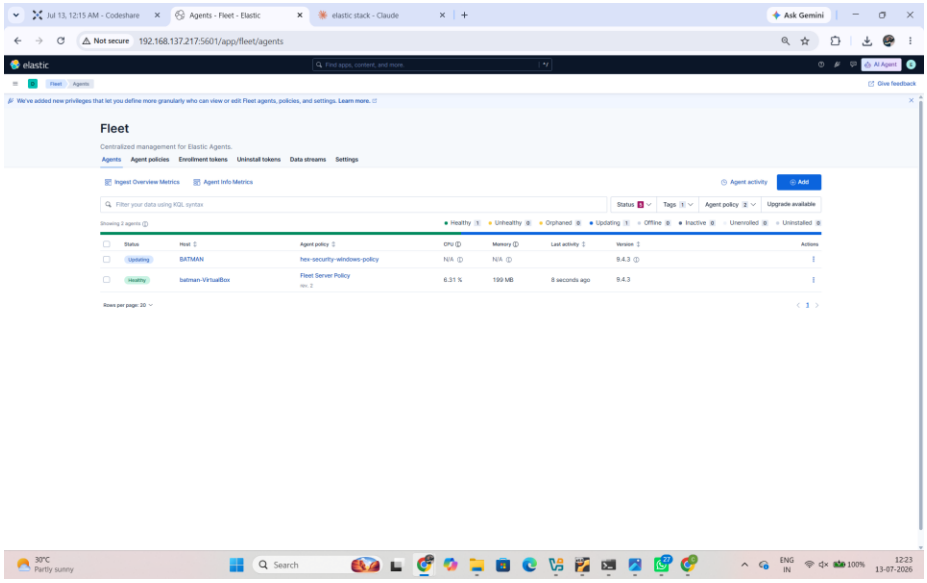

Figure 5. Fleet → Agents immediately after enrollment: BATMAN shows "Updating" during its first policy sync, while Fleet Server remains Healthy.

4.2 Linux Agent Enrollment

A second policy, linux-agent-policy, was created for linux-agent, and the Linux Tar install command was run on that host with the same —insecure flag:

```
curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.4.3-linux-x86_64.tar.gz
tar xzvf elastic-agent-9.4.3-linux-x86_64.tar.gz
cd elastic-agent-9.4.3-linux-x86_64
sudo ./elastic-agent install --url=https://<elk-server-ip>:8220 --enrollment-token=<token> --insecure
```

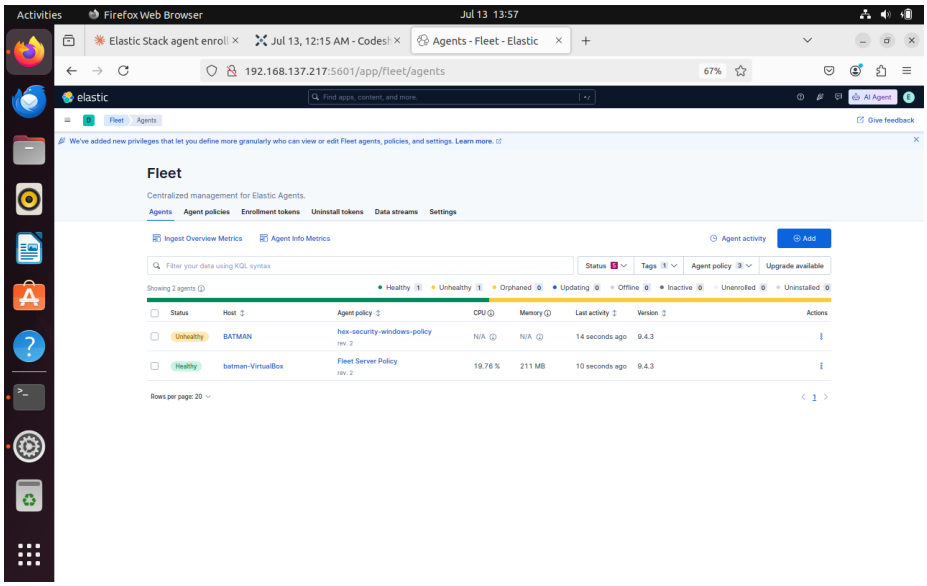

Figure 6. Extraction of the Elastic Agent package on linux-agent immediately prior to running the install command.

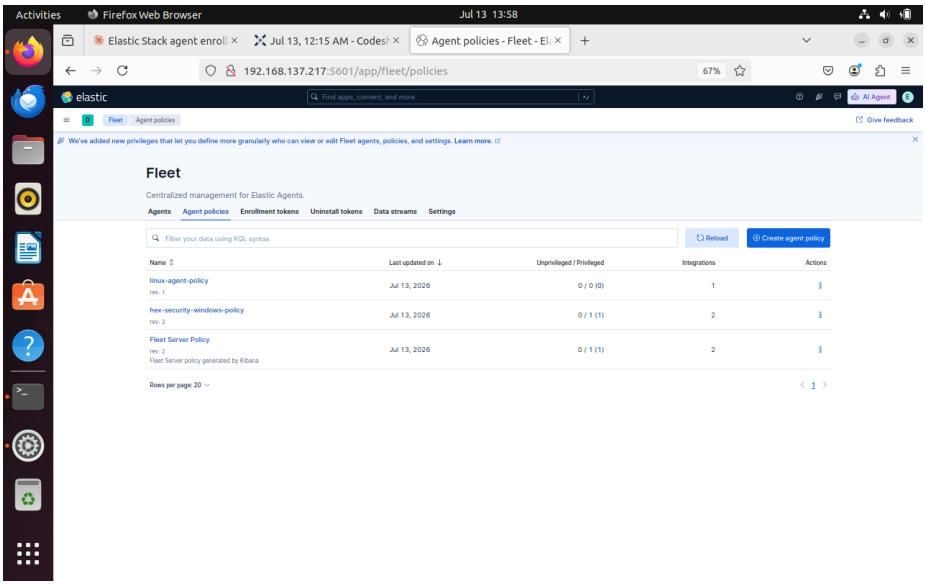

Figure 7. Fleet → Agent policies listing all three policies after enrollment: linux-agent-policy, hex-security-windows-policy and Fleet Server Policy.

5. Endpoint Monitoring Configuration

5.1 Sysmon Visibility on Windows

The course material this lab was originally based on referred to a “Sysmon for Windows” integration package. Searching Kibana's Integrations catalog for it returned no such package — only an unrelated Oracle integration and “Sysmon for Linux”:

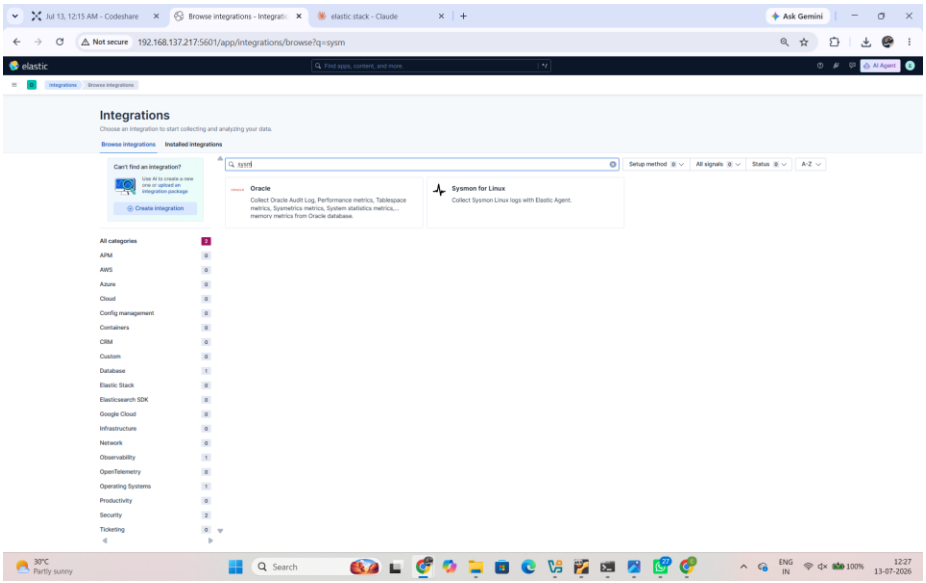

Figure 8. Integrations search for “sysm” returning only Oracle and Sysmon for Linux — confirming no standalone Windows Sysmon package exists in this Elastic version.

Sysmon collection for Windows is in fact bundled as one of several log channels inside Elastic's general-purpose Windows integration. The first attempt at adding it defaulted to creating a new policy rather than attaching to the existing Windows policy, which had to be corrected to the Existing Hosts tab before saving:

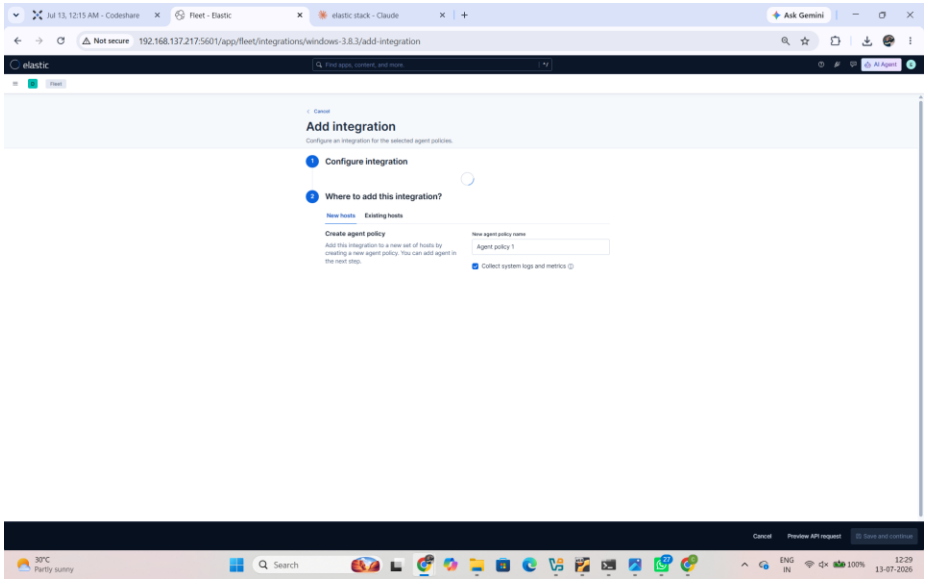

Figure 9. Add Integration screen for the Windows package, showing the default "New hosts" tab — corrected to "Existing hosts" → hex-security-windows-policy.

With the correct policy targeted, the Sysmon Operational log channel was enabled, collecting the Microsoft-Windows-Sysmon/Operational event channel:

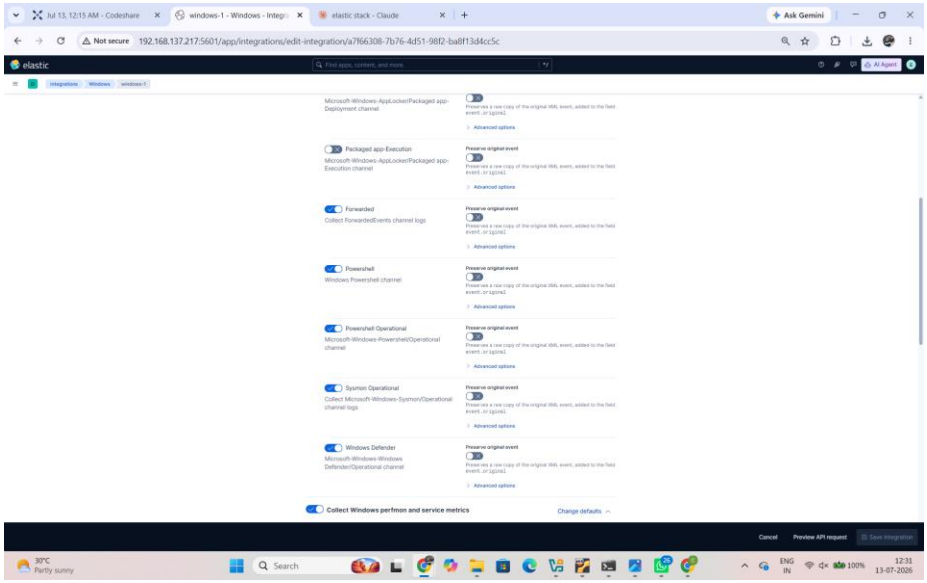

Figure 10. Windows integration configuration with "Sysmon Operational" toggled on.

5.2 File Integrity Monitoring on Linux via auditd

On linux-agent, auditd was installed and enabled independently of the Elastic Agent, and watch rules were added for the two files most commonly tampered with by an attacker establishing persistence:

```
sudo apt install -y auditd audispd-plugins
sudo systemctl enable --now auditd
sudo nano /etc/audit/rules.d/audit.rules
  -w /etc/passwd -p wa -k passwd_changes
  -w /etc/shadow -p wa -k shadow_changes
```

```
sudo systemctl restart auditd
sudo auditctl -l
```

[No screenshot captured for this step: the audit.rules file edit and auditctl -l confirmation — a local terminal step with no Kibana UI equivalent]

6. Infrastructure Troubleshooting

A disproportionate share of this project's effort went into diagnosing why previously working infrastructure had stopped working, rather than into first-time configuration. This section documents that process in detail, since correctly triaging a degraded agent fleet is itself one of the most transferable skills this lab produced.

6.1 Authentication Failure on Fleet Server

After a reboot of elk-server, Fleet Server's own system integration reported Degraded, tracing back to an authentication failure against Elasticsearch:

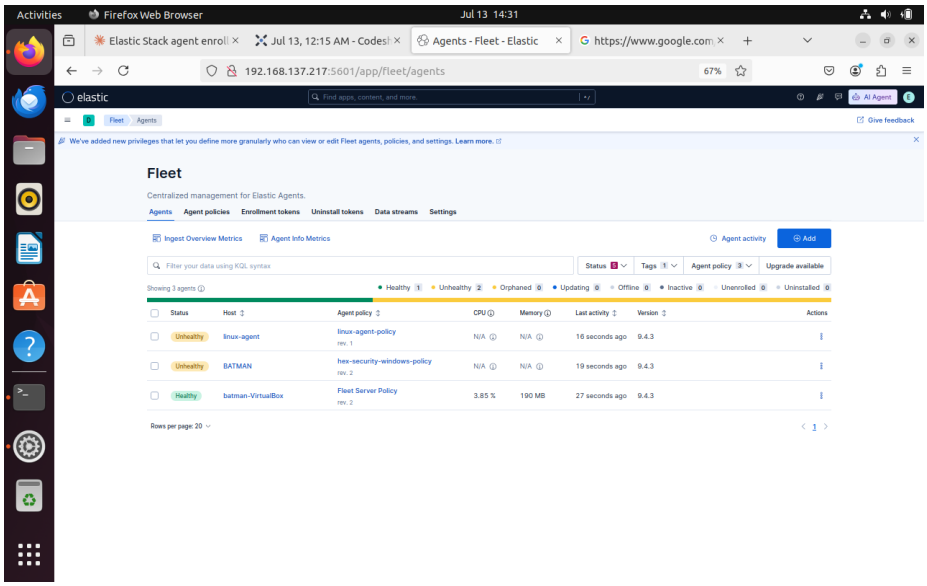

Figure 11. Fleet Server agent details showing the system-1 integration "Needs attention" and Metrics: default "Degraded — Elasticsearch request failed: 401 Unauthorized."

The cause was a drifted elastic superuser password, the result of having been reset multiple times across earlier sessions. It was resolved by repeating the credential reset procedure from Section 3.2 and restarting both core services.

6.2 Both Endpoint Agents Unhealthy

Following the credential incident, both linux-agent and BATMAN reported Unhealthy simultaneously while Fleet Server itself remained Healthy — a pattern indicating one shared upstream cause rather than two independent endpoint failures:

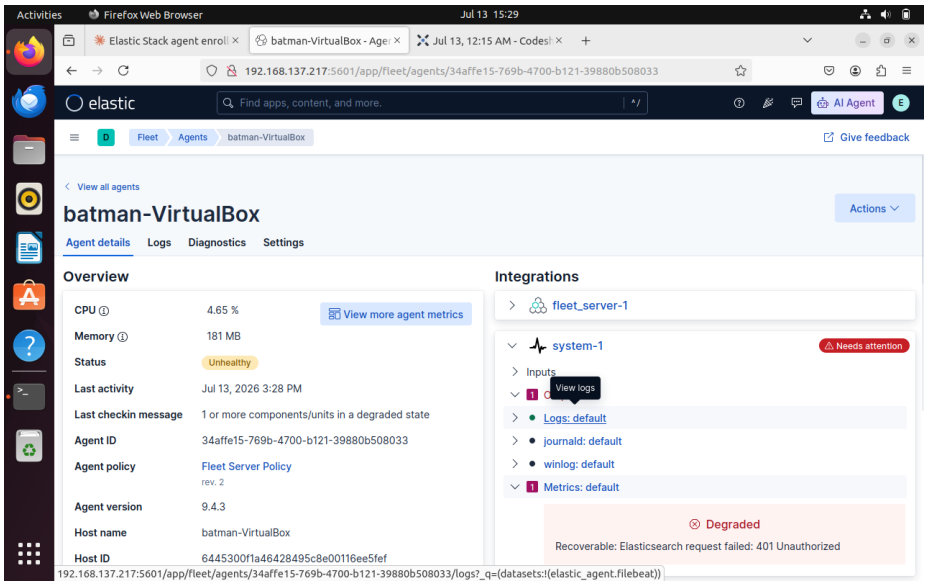

Figure 12. Fleet → Agents showing linux-agent and BATMAN both Unhealthy with N/A CPU/Memory, while Fleet Server remained Healthy.

6.3 Fleet Server Host Pointed at a Stale Address

A subsequent DHCP lease renewal changed elk-server's IP address. The Fleet Server host entry in Kibana still referenced the previous address and had to be corrected before either agent could reconnect:

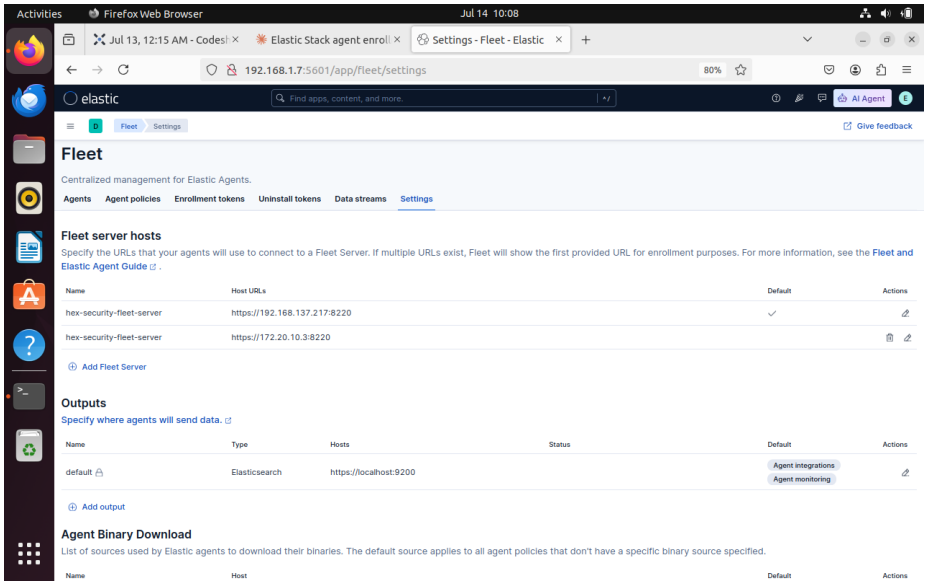

Figure 13. Fleet → Settings → Fleet server hosts, before correction: the default entry still points at the stale address 192.168.137.217:8220.

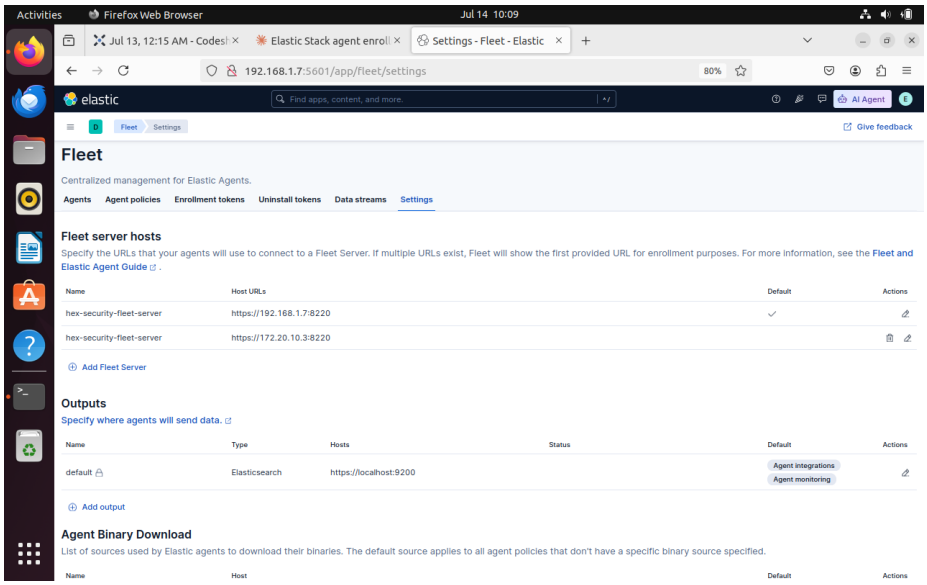

Figure 14. The same screen after correction, with the default Fleet Server host updated to the current address, 192.168.1.7:8220.

Because the address had changed, both endpoint agents also had to be uninstalled and re-enrolled with a freshly generated token against the corrected Fleet Server URL:

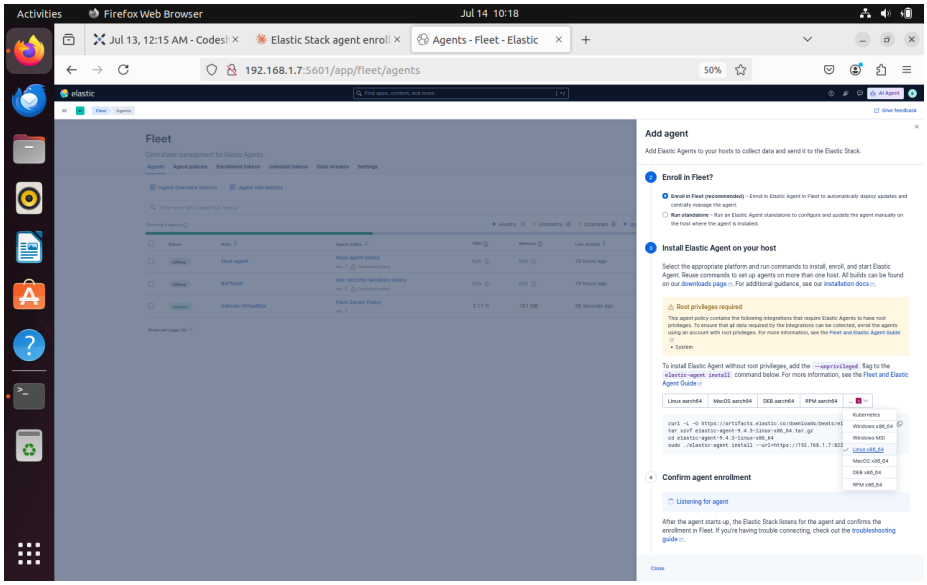

Figure 15. Kibana's Add Agent flyout generating a fresh Linux install command pointed at the corrected Fleet Server address.

6.4 Fleet Output Misrouted to Localhost

Even after re-enrollment, linux-agent continued to report Degraded, with the underlying error dial tcp 127.0.0.1:9200: connection refused. This was a distinct, third root cause: the Fleet default output — the setting controlling where agents actually ship collected data, separate from the Fleet Server URL used only for management — was still configured as https://localhost:9200, a value that only resolves correctly on elk-server itself. Correcting the output host under Fleet → Settings → Outputs to elk-server's real address resolved the issue for both remote agents simultaneously.

7. Attack Simulation and Detection Engineering — SSH Brute-Force (T1110)

7.1 Attack Execution

From the Kali attacker machine, Hydra was run against linux-agent's SSH service using a small local password list targeting a known account:

```
hydra -l batman -P passwords.txt ssh://192.168.1.8
```

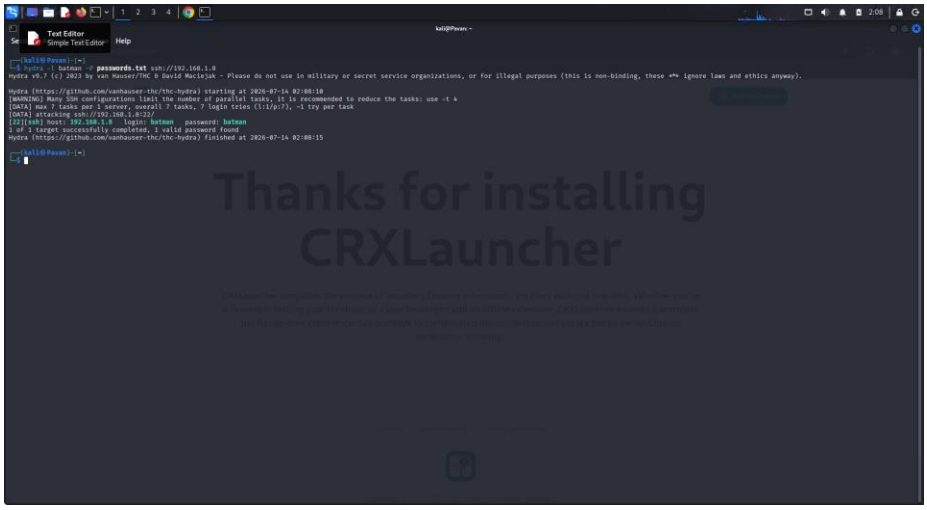

Figure 16. Hydra v9.7 against 192.168.1.8:22, reporting "1 of 1 target successfully completed, 1 valid password found."

7.2 Detection Query

The detection query targets failed sshd authentication events on linux-agent using ECS-normalized fields:

```
event.outcome: "failure" and process.name: "sshd"
```

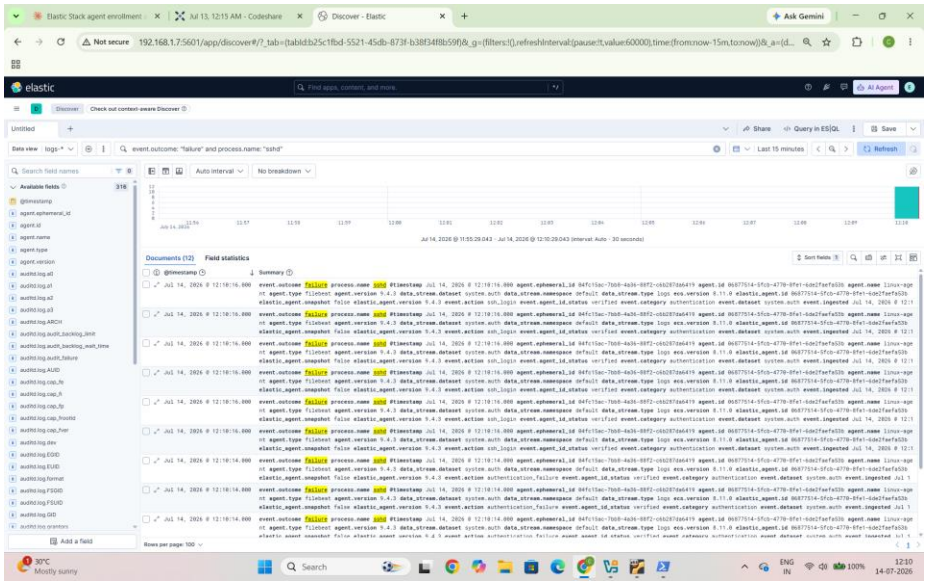

Figure 17. Kibana Discover, last 15 minutes: 12 documents from linux-agent, event.action values ssh_login and authentication_failure — the Hydra attack traffic.

7.3 Extended Validation

The identical query was re-run over a one-year window later in the project, after the infrastructure rebuild described in Section 6, to confirm the detection was reproducible rather than a one-off artifact of the first test:

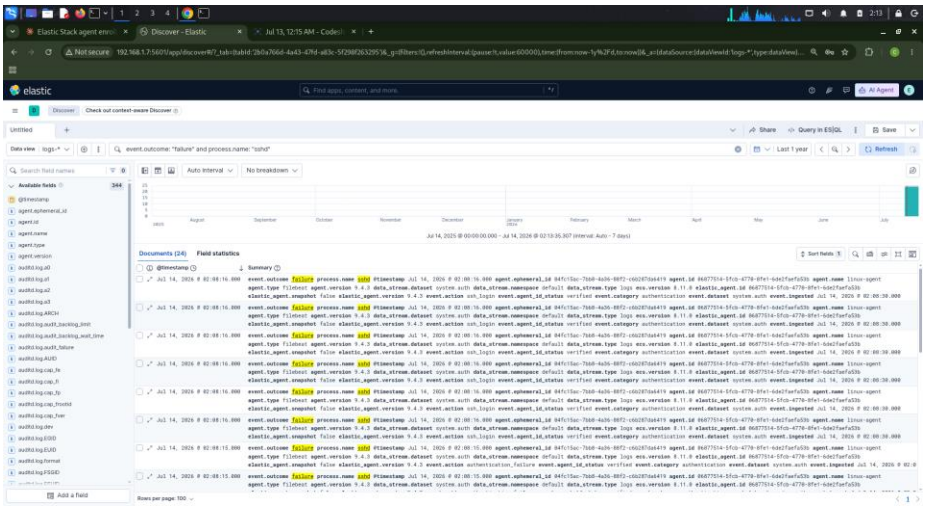

Figure 18. The same query over a one-year range, returning 24 total documents across the full project timeline.

8. Attack Simulation — RDP Brute-Force Attempt and an Environmental Limitation

The equivalent attack was attempted against the Windows endpoint:

```
hydra -l administrator -P passwords.txt rdp://192.168.1.5
```

The attack failed outright with freerdp: The connection failed to establish on every attempt — not a wrong-password failure, but no connection at all. Diagnosis on the Windows host found the Remote Desktop service running but access blocked; after clearing the block and opening the firewall, port 3389 still refused every connection. Checking the Windows edition directly explained why:

```
Get-ComputerInfo | Select-Object WindowsProductName
  -> Windows 10 Home Single Language
```

[No screenshot captured for this step: the RDP connection-failure and Windows-edition-check terminal output — captured as text during the session rather than as a saved image]

Windows 10/11 Home cannot host inbound RDP sessions under any firewall or registry configuration; that capability is restricted to Windows Pro, Enterprise and Education editions. This is an operating-system licensing limitation rather than a misconfiguration, and it could not be resolved within this environment.

To still confirm that the underlying Sysmon telemetry pipeline was functioning, independent of the RDP-hosting limitation, a generic connection probe was run from the attacker machine and checked with a broader query:

```
winlog.channel: "Microsoft-Windows-Sysmon/Operational" and event.code: "3"
```

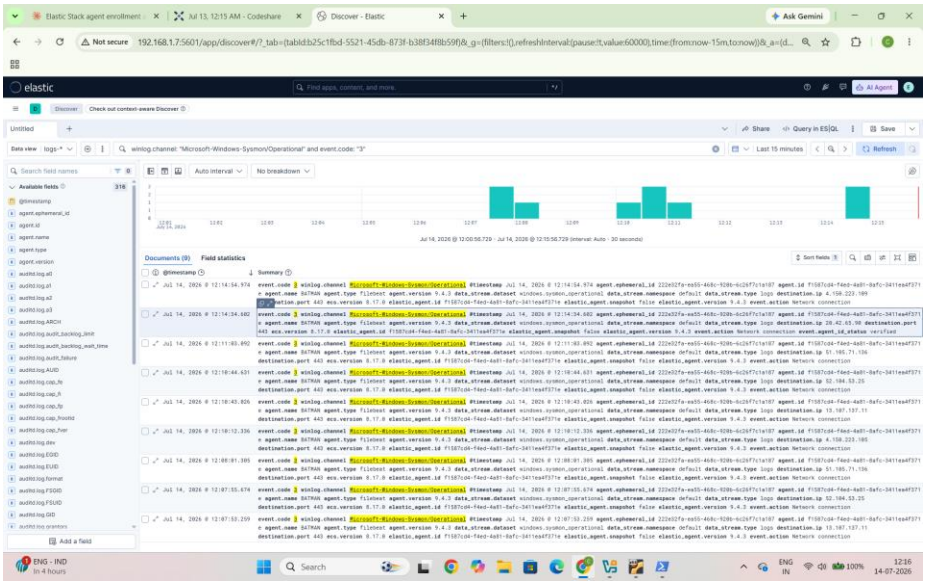

Figure 19. Kibana Discover, last 15 minutes: 9 documents from BATMAN, event.action "Network connection" — confirming Sysmon → Fleet → Elasticsearch ingestion works end to end.

It should be noted precisely what this result does and does not demonstrate: the destination addresses in these events are the Windows host's own background traffic (cloud/telemetry services on port 443), not the attacker's connection attempts, since those were refused before completing a TCP handshake and therefore did not generate a Sysmon Event ID 3 record. This is documented here as pipeline validation, not as a successful RDP attack detection.

9. Attack Simulation — File Integrity Monitoring (auditd)

A rogue account line was appended directly to /etc/passwd on linux-agent to simulate the kind of persistence mechanism an attacker or malicious script might use after gaining a foothold:

```
sudo bash -c 'echo "testuser:x:1500:1500:Test User:/home/testuser:/bin/bash" >>
/etc/passwd'
```

```
auditd.log.key: "passwd_changes"
```

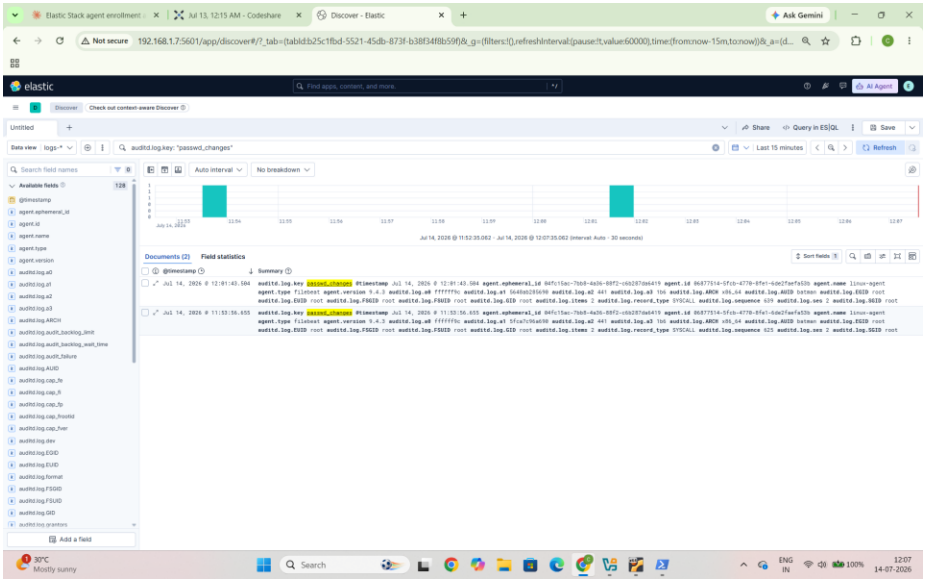

Figure 20. Kibana Discover, last 15 minutes: 2 documents, agent.name linux-agent, record_type SYSCALL — the simulated /etc/passwd tamper captured in real time.

The same query re-run over a one-year window confirmed 7 total matching events across the whole project, including this test and earlier verification runs:

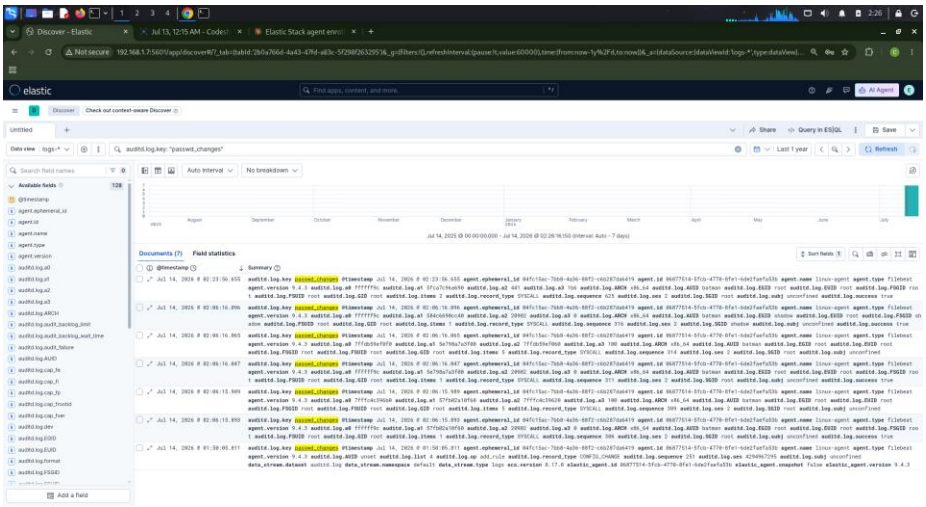

Figure 21. The same query over a one-year range, returning 7 total documents.

10. MITRE ATT&CK Mapping

Activity	Technique ID	Tactic
SSH brute-force against linux-agent (Section 7)	T1110.001 (Password Guessing)	Credential Access
Persistence via rogue account added to /etc/passwd (Section 9)	T1098 / T1136	Persistence
RDP brute-force attempt against windows-agent (Section 8)	T1110.001 / T1021.001	Credential Access / Lateral Movement

Activity	Technique ID	Tactic
Generic TCP connection probing (Section 8)	T1046	Discovery

The RDP row reflects the intended attack technique, not a confirmed detection: as documented in Section 8, the attack traffic itself was not captured due to the Windows Home RDP-hosting limitation, and is distinguished here from the confirmed SSH and auditd mappings above it.

11. Key Findings and Lessons Learned

- A working three-endpoint Fleet-managed pipeline was successfully built, broken by two separate network address changes and a credential-drift incident, correctly diagnosed, and rebuilt — demonstrating that the architecture is recoverable, not merely installable.
- Certificate trust, stale IP configuration, credential drift and output misrouting produced the identical "agent Unhealthy" symptom in the Fleet UI; correctly triaging a monitoring outage required checking each of these in turn rather than assuming a single cause.
- Vendor and third-party course material can reference discontinued or renamed components — this project's original source material referred to a "Sysmon for Windows" package that no longer exists as such — and verifying current product behavior before committing configuration time is necessary.
- SSH brute-force and auditd detections were each validated twice: immediately after the triggering action, and again over a much longer time window, which is stronger evidence than a single screenshot since it demonstrates the detection survived a full infrastructure rebuild.
- Not every planned attack scenario can be completed exactly as scoped in a real, physically-constrained lab; the Windows Home RDP-hosting limitation is an OS licensing restriction, not a configuration error, and is documented here as such rather than omitted.

LinkedIn: pavankumar022
GitHub: pavankumar022
E-mail: pavankumar797524@gmail.com